

DDoS / Load-Flood Test Strategy

DDoS / Load-Flood Test Strategy

Revision: 2 Last modified: 2026-06-26T12:00:00Z

Reconciled (§11.4.35, 2026-06-26): the `wg_init_flood` generator (§2, §8) now targets the **canonical plain-WG port gw:51820** — a real WG MessageInitiation flood hits the plain-WG port that test-rig.md §3.1 drops, not the MASQUE/QUIC :443 port (the earlier gw:443 target conflated the two surfaces). The volumetric iperf3 flood keeps :443 (the MASQUE/QUIC surface); the edge multiplexes both, but each flood class is modelled on its own port for rig fidelity.

Master technical specification — Volume 8 (Testing & QA), nano-detail document **ddos**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens 10-testing-acceptance-and-qa.md §5.8 (DDOS) and the §2 taxonomy row marked NOT_APPLICABLE: single-node-selfhost at MVP into an implementation-ready, **author-now-parked** (decision QA-D2, overview §10) bank that re-arms mechanically when Phase 2's public multi-tenant surface exists. SPEC-ONLY: it describes the harness, the fixtures, the captured evidence, the acceptance gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — [OVERVIEW] = doc 10; [TM] = [../v05-security/threat-model.md]; [01-DP] = doc 01 data plane; [04_P1]/[04_P2] = the Phase-1/Phase-2 refined docs; [svc-policy]/[svc-pki] = the Volume-3 nano-docs. Any claim not grounded in the evidence base is marked **UNVERIFIED** per constitution §11.4.6 — never fabricated.

Table of contents

- 0. Scope on HelixVPN surfaces + MVP status
- 1. The two threats this suite proves bounded
- 2. Harness — the flood rig
- 3. Fixtures — real, no mocks (§11.4.27)

- 4. Captured evidence (§11.4.69 / .85)
 - 5. Determinism (§11.4.50)
 - 6. Acceptance gate
 - 7. The paired §1.1 mutation
 - 8. Test skeletons
 - 9. The MVP SKIP record + re-arm trigger
 - Sources verified
-

0. Scope on HelixVPN surfaces + MVP status

A VPN gateway is, by definition, an Internet-reachable target: the **Rust edge** listens on :443 for WG/MASQUE datagrams ([TM §5.3], T-EDGE-D-1) and the **Go control plane** answers enroll / WatchNetworkMap / policy RPCs over mTLS ([TM §5.2]). DDoS is the threat that *no single client* and *no flood of init packets* may exhaust those surfaces. At MVP the deployment is a single rootless-Podman host with no public multi-tenant surface [04_P1 §11]; a volumetric DDoS at MVP would exercise a threat the topology does not yet present, so the coverage-ledger cell is NOT_APPLICABLE: single-node-selfhost (§11.4.6, overview §6.3 row F-DDOS-FLOOD) — an **honest, reasoned** NA, never a silent omission.

The seam, however, is built **now** so the suite re-arms by data, not by code: the per-tenant / per-source rate-limit token buckets in Redis ([TM §5.1 T-CONN-D-1], [svc-pki §3.4]), WG's stateless cookie-reply anti-DoS ([01-DP], [TM §5.3 T-EDGE-D-1]), and the stateless fail-static edge that drops and rootless-auto-restarts rather than crashing [01-DP I3]. This document authors the bank against those seams; QA-D2 records the decision (author-now-parked).

Surface	Flood class	MVP seam (built)	Phase-2 re-arm
edge :443 data port	WG/MASQUE handshake-init flood	WG cookie-reply; host firewall floor	full handshake- flood bank
edge :443 data port	volumetric byte flood	stateless drop + rootless restart [01-DP I3]	upstream- scrubbing- assisted bank
control plane mTLS RPC	enroll / SignDeviceCert flood	Redis token bucket, per- tenant + per-source [svc-pki §3.4]	KMS-quota- protection bank

Surface	Flood class	MVP seam (built)	Phase-2 re-arm
control plane	WatchNetworkMap stream-open flood	bounded streams [svc-policy §11]	coordinator fan-out soak
control plane	policy compile-bomb	bounded 0(devices×rules×targets) [TM §5.2 T-CP-D-1]	10k×1k tenant soak

1. The two threats this suite proves bounded

The bank is built around two falsifiable claims, each phrased so the PASS is a captured-evidence proof that a **legitimate** client is unharmed *while the flood runs* (anti-bluff: a flood test that only proves "the flood happened" is a §11.4 bluff — it must prove the service stayed usable).

- **D-FLOOD-HANDSHAKE.** A torrent of WG/MASQUE init packets (and a torrent of unauthenticated enroll attempts) must not exhaust the gateway. **Claim:** during the flood, a *legitimate* enrolled client's handshake p99 latency stays under its SLO budget, the edge process never OOM/crashes (`helix_edge_up == 1` throughout), and `wg_init_dropped` / `ratelimit_rejected` counters rise (proving the rate limiter *engaged*, not that the flood was absent).
- **D-FLOOD-VOLUMETRIC.** A byte flood on :443 must make the stateless edge **fail-static** — drop excess, never crash, rootless-auto-restart if the supervisor cycles it [01-DP I3]. **Claim:** the edge liveness counter stays 1 (or restarts within the supervisor budget with zero plaintext leak during the gap — composes the kill-switch S9 evidence, [TM §5.4 T-CLI-T-1]).

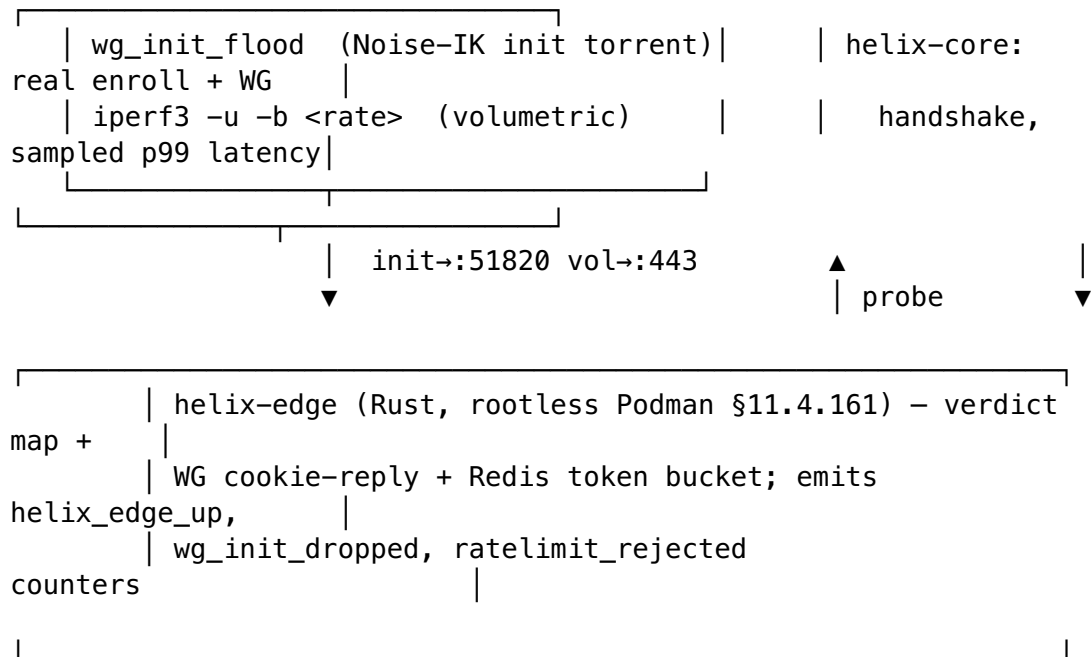
Each claim is a challenges bank entry (overview §5.5) scored on captured evidence, never the generator's exit code.

2. Harness — the flood rig

The rig extends the netns rig (overview §5.3, [test-rig.md]) with two flood generators and a **legitimate-client probe** that runs *concurrently with* the flood — the probe is the thing the acceptance gate measures. Single-resource-owner partitioning (§11.4.119): exactly one process owns the edge data port; the flood generators and the legit probe are distinct netns peers.

```

netns: flooders (N attacker peers)
legit (1 honest client)
netns:
```



- **wg_init_flood** — a custom Noise-IK initiation-message generator (not a mock: it emits real WG MessageInitiation frames with valid framing but unregistered static keys, exactly the cheapest attacker packet [01-DP], [TM T-EDGE-D-1]); it targets the **canonical plain-WG port udp/51820** (the port [test-rig.md §3.1] drops), *not* the MASQUE/QUIC :443 surface — plain WG is udp/51820, MASQUE/QUIC is :443 (the edge multiplexes both, but the handshake-flood models real plain WG on its own port). Rust binary rig/wg_init_flood, rate configurable (--pps).
- **iperf3 -u -b <rate>** — UDP volumetric flood at line rate toward the MASQUE/QUIC :443.
- **legit probe** — a real helix-core instance performing genuine enroll + WG handshake + iperf3 goodput in the legit netns, sampling handshake completion latency.
- **counter scrape** — helix-edge's Prometheus endpoint (control-plane-local, no per-user data, invariant I5) is scraped at 1 s cadence into a CSV.

The rig runs under containers/pkg/boot for Redis (the token-bucket backing store) — the sole container path (§11.4.76); the netns peers need CAP_NET_ADMIN (the documented scoped sudo exception, overview §9), never container-management escalation.

3. Fixtures — real, no mocks (§11.4.27)

DDoS is an integration/e2e-class type: **no mocks** below the unit layer. Fixtures are real:

Fixture	What it is	Why real
legit_client.toml	a genuinely enrolled helix-core config (real device cert, real WG key)	the probe must complete a <i>real</i> handshake — a mocked client cannot prove the limiter spares legitimate traffic
attacker_keys.bin	N=10k unregistered Curve25519 static keys for wg_init_flood	real WG frames with keys the edge will reject — the cheapest real attacker packet
tenant_ratelimit.yaml	the real Redis token-bucket config under test [svc-pki §3.4]	the limiter's real thresholds, not a stubbed allow-all
edge_image	the published rootless-Podman edge image (S8)	the suite must flood the <i>shipping</i> artifact (§11.4.108), not a debug build

The legitimate-client fixture is the load-bearing one: the suite's entire value is proving the **real** honest client is unharmed; a stubbed probe would be a B3 wrong-plane bluff.

4. Captured evidence (§11.4.69 / .85)

Every PASS cites artifacts under qa-results/ddos/<run-id>/, scored by the challenges engine, never the generator's exit code. The §11.4.69 evidence class is network_throughput / network_connectivity for the legit probe and a liveness counter for the edge.

Artifact	Shape	Asserts
legit_latency.json	per-sample handshake-complete + goodput series, p50/p95/p99	legit p99 ≤ budget <i>during</i> the flood
edge_liveness.csv	1 Hz helix_edge_up, RSS, FD count	edge never 0 (or restart within supervisor budget)
limiter_counters.csv	wg_init_dropped, ratelimit_rejected time series	the limiter <i>engaged</i> (counters rose) — proves the flood reached the edge,

Artifact	Shape	Asserts
gap_pcap.pcap	host-iface capture during any edge restart	not that it was absent
		zero plaintext / zero :53 during the gap (composes S9, [TM T-CLI-T-1])
categorised_errors.txt	per-error-class counts from the flood window (\$11.4.85)	refused-cleanly vs crashed classification

Anti-bluff rule (overview §0, B2): "no error during flood" is **not** evidence — the limiter counters MUST show engagement and the legit-probe latency CSV MUST show bounded service, or the test is a B2 absence-of-error bluff and FAILs.

5. Determinism (§11.4.50)

The flood is inherently noisy, so determinism is asserted on the **verdict**, not the raw rate. `ab_run_n_times "ddos-handshake" 3 run_handshake_flood` runs the bank N=3 against the same edge image MD5 + same rig; the per-run evidence-hash is computed over the *verdict tuple* (`edge_stayed_up: bool`, `legit_p99_within_budget: bool`, `limiter_engaged: bool`, `leak_during_gap: bool`) — all three runs MUST yield the identical verdict tuple. A run where the edge stays up in 2 of 3 is **auto-FAIL** (§11.4.50: no first-pass-was-a-flake escape). Cycle-validation runs N=10.

The raw rate/latency numbers are recorded for trend (§11.4.24 min/max/mean/p95) but are not the determinism key — the *verdict* is, because the property under test is binary ("bounded / not").

6. Acceptance gate

#	Bar	Evidence	Phase
DDOS-G1	legit handshake p99 ≤ SLO budget while flood at ≥ design-attack-rate runs	legit_latency.json	Phase 2
DDOS-G2	helix_edge_up == 1 throughout, OR restart ≤ supervisor budget with zero leak in gap_pcap	edge_liveness.csv + gap_pcap.pcap	Phase 2

#	Bar	Evidence	Phase
DDOS-G3	limiter engaged: ratelimit_rejected/ wg_init_dropped strictly > 0 during flood	limiter_counters.csv	Phase 2
DDOS-G4	control-plane enroll-flood: legit enroll still succeeds; KMS SignDeviceCert quota not exhausted	enroll-success log + quota counter	Phase 2

At MVP the gate is recorded NOT_APPLICABLE: single-node-selfhost in the coverage ledger with the §11.4.3 reason; the gate is **release-blocking in Phase 2** when the multi-tenant surface exists. A gate that cannot clear in its time box **is the finding** — escalate per §11.4.66, never overrun silently.

7. The paired §1.1 mutation

The mutation proves the gate is not a bluff — it must FAIL the gate when the protection is removed:

MUTATION (paired §1.1, gate CM-DDOS-RATELIMIT-ENGAGED):

Disable the Redis token-bucket check in the edge enroll/
handshake path

(the `ratelimit.Allow()` call returns true unconditionally).

EXPECTED: under the flood, the legit probe's p99 blows past
budget AND

limiter_counters.csv shows ZERO ratelimit_rejected
→ DDOS-G1 + DDOS-G3 FAIL → mutation caught.

RESTORE: re-enable ratelimit.Allow(); re-run → GREEN.

A second mutation (CM-DDOS-EDGE-FAILSTATIC) replaces the stateless drop with an unbounded allocation per init packet; expected: edge RSS climbs to OOM under the flood, helix_edge_up drops with no clean restart → DDOS-G2 FAILs → caught. Both mutations are restored and the working tree is verified quiescent (§11.4.84) before any commit.

8. Test skeletons

```
// rig/wg_init_flood/src/main.rs - REAL Noise-IK init torrent (not
// a mock), §11.4.27
fn main() -> anyhow::Result<()> {
    let args = Args::parse(); // --target :443 --
    // pps --keys attacker_keys.bin
    let keys = load_static_keys(&args.keys)?; // 10k unregistered
    // Curve25519 keys
```

```

let sock = UdpSocket::bind("0.0.0.0:0");
let pacer = Pacer::new(args.pps);
loop {
    let k = keys.next_round_robin();
    let init = wg::MessageInitiation::new(&k,
&args.gateway_pubkey); // valid framing, bad identity
sock.send_to(&init.encode(),
args.target)?; // the cheapest
attacker packet
pacer.tick(); // bounded send
rate; never floods the *host* (§12 host-safety)
}
}

# rig/ddos_handshake.sh – the D-FLOOD-HANDSHAKE challenge driver
set -euo pipefail
out="qa-results/ddos/$(date +%s)"; mkdir -p "$out"
trap 'rig/flood_stop.sh; rig/netns_down.sh' EXIT #
    §11.4.14 cleanup on every path
scrape_counters_1hz "$out/limiter_counters.csv" &
    # edge counters
sample_edge_liveness_1hz "$out/edge_liveness.csv" & #
    helix_edge_up, RSS
ip netns exec flooders rig/wg_init_flood --target gw:51820 --pps
"$ATTACK_PPS" --keys attacker_keys.bin & # plain-WG port
(test-rig.md §3.1)
sleep 2
    # let the flood ramp
ip netns exec legit helix-core probe --enroll legit_client.toml --
iperf 20 \
    --latency-out "$out/legit_latency.json" # the
    LEGIT client, concurrently
p99=$(jq '.handshake_p99_ms' "$out/legit_latency.json")
up=$(min_col "$out/edge_liveness.csv" helix_edge_up) # min
    over the window
rej=$(max_col "$out/limiter_counters.csv" ratelimit_rejected)
[ "$up" = 1 ] && within_budget "$p99" && [ "$rej" -gt 0 ] \
    && ab_pass_with_evidence "edge bounded under handshake flood;
    legit p99=$p99ms" "$out" \
    || ab_fail "DDoS bound breached: up=$up p99=$p99 rejected=$rej"

# challenges/banks/helixvpn_phase2_ddos.yaml – parked, re-arms in
    Phase 2 (QA-D2)
bank: helixvpn-phase2-ddos
arm_condition: deployment.topology == "multi-tenant-ha" #
    mechanical re-arm, not a code edit
challenges:
- id: HVPN-CHAL-DDOS-HANDSHAKE
  feature_class: network_connectivity
  driver: rig/ddos_handshake.sh
  evidence:
    latency: { path: "legit_latency.json", assert:
        "handshake_p99_ms <= slo.handshake" }
    liveness: { path: "edge_liveness.csv", assert_min:
        "helix_edge_up == 1" }

```



```
limiter: { path: "limiter_counters.csv", assert:
  "max(ratelimit_rejected) > 0" } # defeats B2
self_validated: true #
golden-bad: limiter-off run MUST score FAIL
```

9. The MVP SKIP record + re-arm trigger

At MVP the coverage ledger carries (overview §6):

```
feature_id = F-DDOS-FLOOD
  DDOS cell: state = NOT_APPLICABLE, skip_reason = 'single-node-
selfhost' -- §11.4.3 closed reason
  CHAL cell: state = NOT_APPLICABLE, skip_reason = 'single-node-
selfhost'
```

The re-arm is **mechanical** (overview §6.2 state machine, §7.3): when the deployment topology changes to multi-tenant-ha in Phase 2, the arm_condition flips the cell from NOT_APPLICABLE to REQUIRED, the parked bank activates, and the gate becomes release-blocking. No code change is needed to re-arm — the bank already exists (QA-D2), satisfying the §11.4.6 "the seam exists now" discipline so Phase-2 cannot ship a DDoS gap by omission.

UNVERIFIED. The concrete design-attack-rate (ATTACK_PPS), the legit-handshake SLO budget under flood, and the supervisor restart budget are Phase-2 measured numbers, not yet results; they are stated as Phase-2 targets here and become facts only when the Phase-2 rig produces the CSVs. The WG cookie-reply effectiveness against the specific init-flood rate is asserted from the WireGuard design [01-DP] but **UNVERIFIED** for HelixVPN's edge until the Phase-2 bank runs.

Sources verified

- [OVERVIEW] [../10-testing-acceptance-and-qa.md] — §2 taxonomy (DDOS NOT_APPLICABLE row), §5.8 (DDoS strategy), §6.3 (F-DDOS-FLOOD ledger row), §7.3 (Phase-2 re-arm), §10 QA-D2. (Read 2026-06-26.)
- [TM] [../v05-security/threat-model.md] — T-EDGE-D-1 (handshake/UDP flood), T-CONN-D-1, T-CP-D-1 (compile-bomb), T-PKI-D-1 (KMS quota), §10 R-DOS residual. (Read 2026-06-26.)
- [01-DP] final/01-data-plane.md — WG stateless cookie-reply anti-DoS, stateless fail-static edge (I3); cited via the overview + threat-model.
- [svc-pki] [../v03-control-plane/svc-pki.md] / [svc-policy] [../v03-control-plane/svc-policy.md] — §3.4 enroll rate limits, §11 bounded compile/streams; cited via the threat-model.

- [04_P1 §11] single-node MVP topology (the basis for the MVP NOT_APPLICABLE); [04_P2] Phase-2 HA surface — cited via the overview.
- Constitution: §11.4.169 (test-type mandate), §11.4.3 (honest SKIP/NA reason), §11.4.6 (no-guessing / UNVERIFIED), §11.4.27 (no-fakes), §11.4.69 (sink-side evidence), §11.4.85 (stress/chaos taxonomy adjacency), §11.4.50 (determinism), §11.4.84 (quiescence), §11.4.119 (single-resource-owner), §11.4.108 (flood the shipping artifact), §1.1 (paired mutation).